VECTOR DATABASES · NOTEBOOK 01 OF 03

01 — Why Vector Databases? The Story, and How to Choose a Model

A guide to Part0_Vector_Databases_Final.ipynb: one customer question a keyword search can't answer, the chain of ideas that answers it — and how to choose the embedding model at the centre of it.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Stack: OpenAI `text-embedding-3-small` · Pinecone serverless (AWS us-east-1)
Index: `ecom-kb-tutorial` · Namespace: `help-articles` · 5 help articles · about 2½–3 hours

1. Where 01 sits
2. The story
3. Chapter 1 — The problem
4. Chapter 2 — Why a keyword search misses it
5. Chapter 3 — Similarity search
6. Chapter 4 — Scalars
7. Chapter 5 — Vectors
8. Chapter 6 — Embeddings
9. Choosing an embedding model ★
10. Chapter 7 — Cosine similarity
11. Chapter 8 — The vector database
12. Chapter 9 — Back to the customer
13. The code, cell by cell
14. Key takeaways and knowledge check

1 Where 01 sits

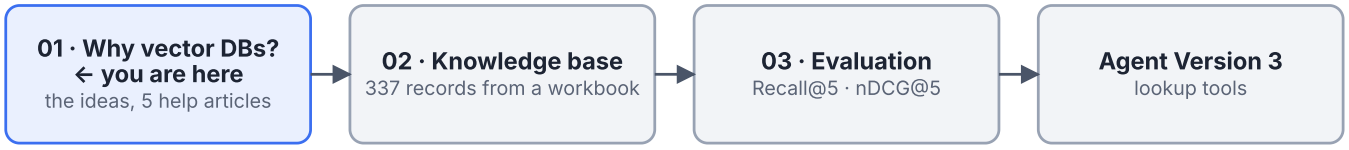


Figure 1. The series. 01 builds the ideas; 02 builds the knowledge base the agent searches.

Three notebooks teach vector databases on one Pinecone index. **01** builds the ideas with five help articles. **02** loads a real knowledge base — the description of an e-commerce database — for an AI SQL assistant. **03** measures how well the search works. Agent Version 3 then searches what 02 loaded.

No machine-learning or linear-algebra background is needed. Python is used only to call the embedding model and Pinecone.

> **Learning goal:** explain in your own words how text becomes an embedding, and how a vector database uses it to retrieve relevant records — and be able to say why we chose the embedding model we use.

2 The story

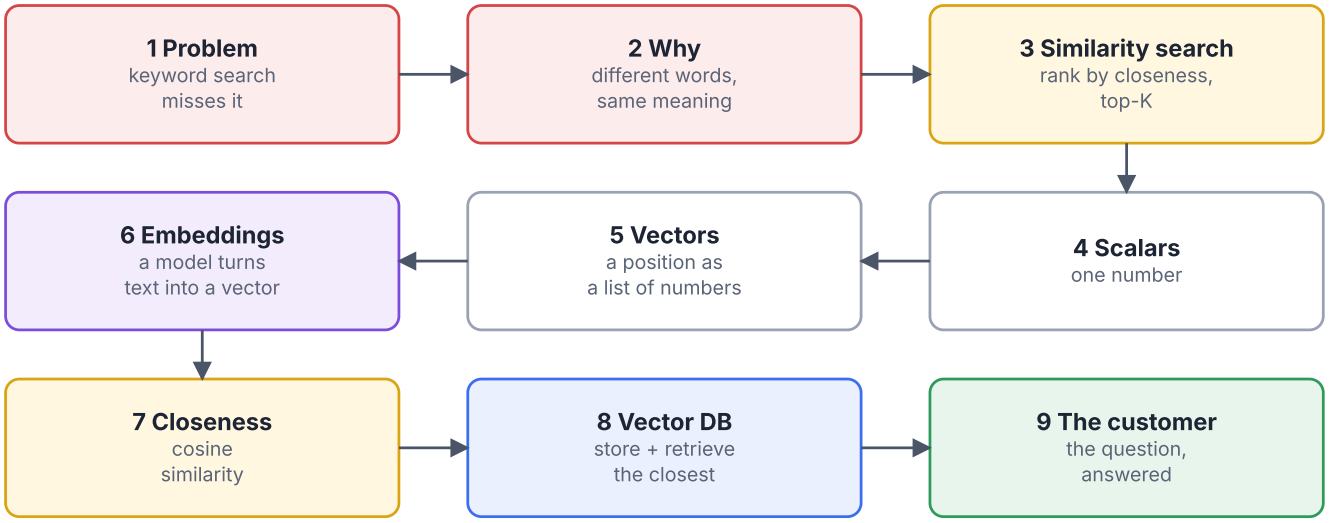


Figure 2. The story. Each chapter answers the question the previous one raised.

Teach 01 as one story. Never introduce an idea before students feel the need for it: each chapter ends with a **next question**, and the next chapter answers it.

Chapter	Question it answers	Time	Code
1–3	Why can't a keyword search find the answer? What do we need instead?	35 min	—
4–5	How do we describe something with numbers?	20 min	—
6	How does text become numbers? Which model?	45 min	setup · embed
7	How do we measure closeness?	15 min	—
8–9	Where do we keep embeddings, and how do we search them?	40 min	store · search
—	Try your own, knowledge check	20 min	try

3 Chapter 1 — The problem

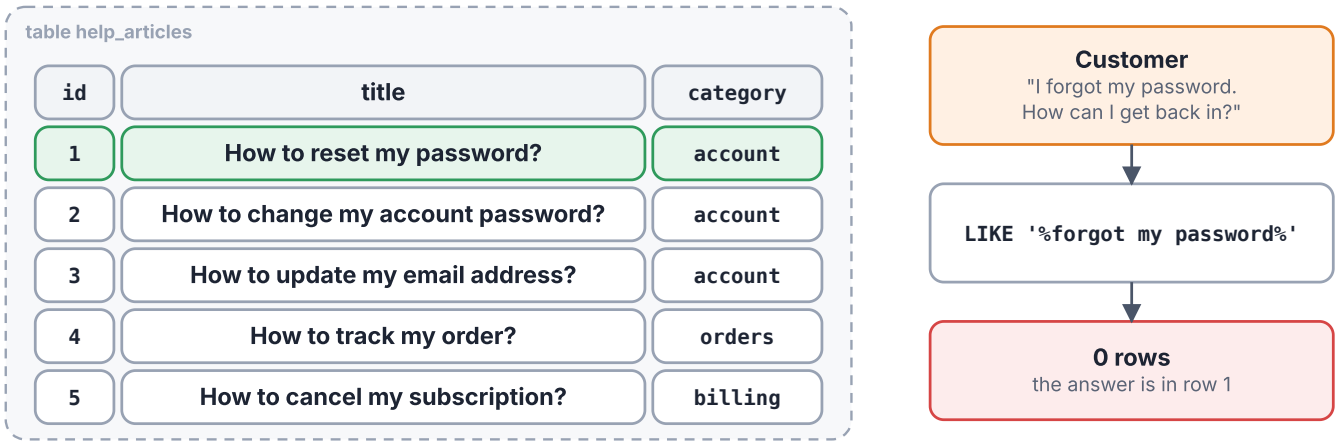


Figure 3. The help articles in a relational table. The answer is in row 1; SQL returns nothing.

A company keeps its help articles in a normal relational table. A customer types **"I forgot my password. How can I get back into my account?"** The search box runs:

```
SELECT * FROM help_articles WHERE title LIKE '%forgot my password%';
-- 0 rows
```

0 rows. The answer is in row 1 — it says "reset", not "forgot". Let students feel this before explaining anything.

4 Chapter 2 — Why a keyword search misses it

`LIKE '%forgot my password%'` asks: does the title contain these characters, in this order? Real customers describe one problem in many ways:

Customer writes	Simple keyword search	What happens
I forgot my password	`LIKE '%forgot%'`	misses row 1 — it says "reset"
I can't log in anymore	`LIKE '%log in%'`	may miss row 1 — those words are absent
I'm locked out of my account	`LIKE '%account%'`	may find an account article — not necessarily the right one

Relational databases are great for	A different kind of question
`WHERE id = 1`	which article **means** something similar to this question?
`WHERE category = 'account'`	which article is most relevant to what the customer meant?
totals, joins, ranges, exact values	find relevant content even when the wording differs

Be precise: the database isn't broken, and relational databases can do full-text search, fuzzy matching and — with extensions such as PostgreSQL `pgvector` — vector search. The narrower point: **a basic keyword query doesn't understand meaning.**

Next question: not only "which row **contains** these words?" but "which rows are most **similar in meaning**?"

5 Chapter 3 — Similarity search

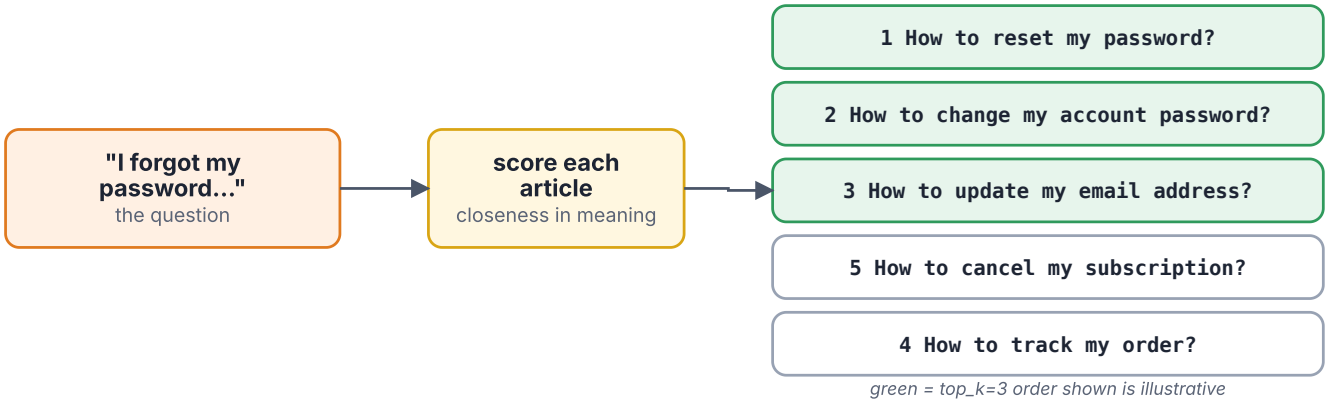


Figure 4. Similarity search: score every article by closeness in meaning, sort, keep the top-K.

Searching for the items closest in meaning is **similarity search**: compare the question with every article, rank closest first, return the **top-K**.

Rank	Help article	Score (made up)
1	How to reset my password?	0.92
2	How to change my account password?	0.88
3	How to update my email address?	0.71
4	How to cancel my subscription?	0.35
5	How to track my order?	0.21

These scores are **made up** to explain ranking. `top_k=1` returns the best one; `top_k=3` the best three. But how can a computer compare meaning? First, each text needs to be written as numbers.

> **Next question:** how do we represent something using numbers?

6 Chapter 4 — Scalars

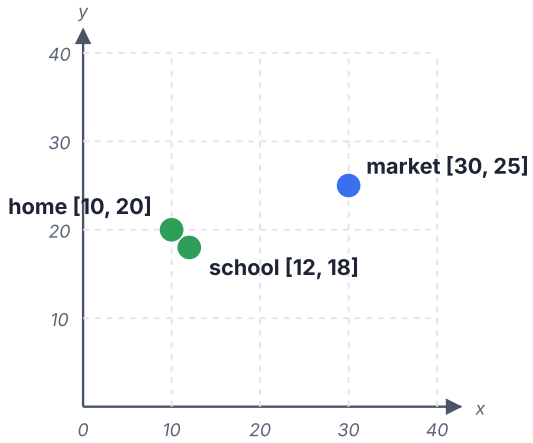


Figure 5. A scalar is one number. Each box holds exactly one value.

A **scalar** is one number: `age = 30`, `price = 500`. That is all we need about scalars — five minutes, no more. It exists so that "vector" has something to be compared with.

> **Next question:** what if one number is not enough to describe something?

7 Chapter 5 — Vectors



home = [10, 20]
10 to the right, 20 up

home ≈ school
similar numbers, close together

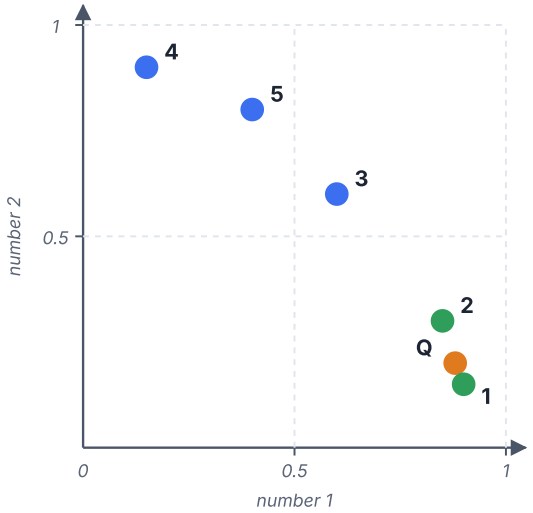
Figure 6. Two numbers give a position. Similar numbers → close together.

A place on a map needs two numbers: `home = [10, 20]` is 10 across and 20 up. Home and school have nearby numbers, so they are close; the market is far.

Word	Meaning	Example
Scalar	one number	`30`
Vector	a list of numbers that represents something	`[10, 20]`
Dimension	how many numbers are in the vector	2 — and `[90, 85, 80]` has 3

Next question: can we represent a **sentence** using a list of numbers too?

8 Chapter 6 — Embeddings



Q *I forgot my password... [0.88, 0.20]*
1 *How to reset my password? [0.90, 0.15]*
2 *How to change my account password? [0.85, 0.30]*
3 *How to update my email address? [0.60, 0.60]*
4 *How to track my order? [0.15, 0.90]*
5 *How to cancel my subscription? [0.40, 0.80]*

made-up numbers, chosen by hand

Figure 7. If text had positions: similar meaning would sit close together (made-up numbers).

Figure 7 shows the goal with numbers **we chose by hand**: the question Q sits next to the password articles. Real numbers come from an **embedding model**:

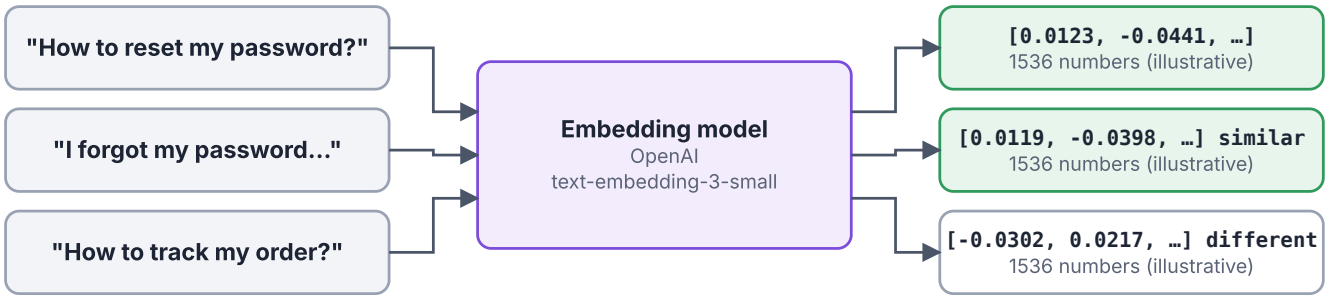


Figure 8. The embedding model turns each text into 1536 numbers. Similar meaning → similar numbers.

- An **embedding** is a vector generated from text by an embedding model.
- **We don't choose the numbers** — the model, trained beforehand on a very large amount of text, does.
- **We don't know what one dimension means.** The information is spread across the whole vector.
- Similar meaning **often** gives similar embeddings — a learned pattern, not a guarantee.
- `text-embedding-3-small` returns **1536** values by default; other models return other sizes.

Tokens and the input limit

A **token** is a piece of text the model reads: a word, part of a word, or punctuation. Every model has a maximum number of tokens per text — and providers behave differently past it:

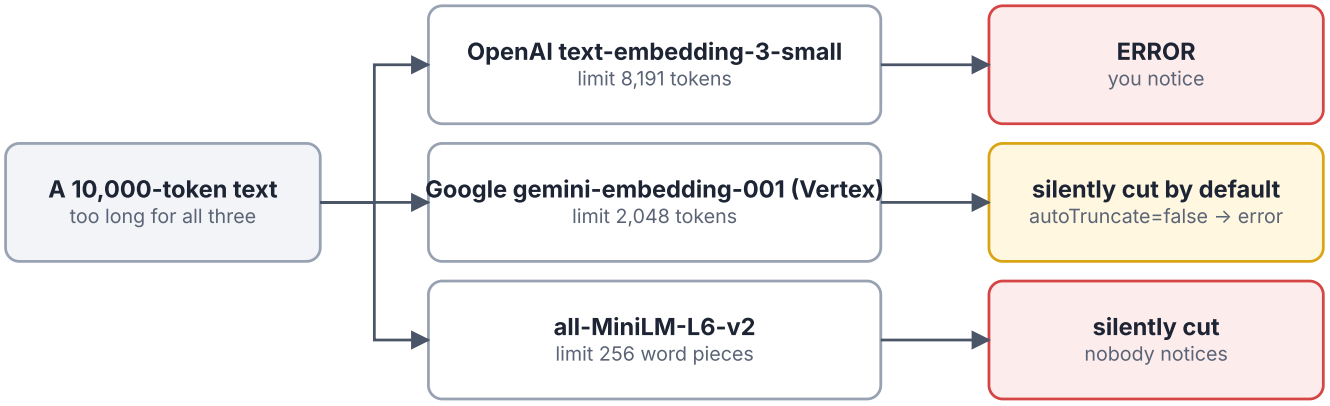


Figure 9. Text longer than the limit: three providers, three behaviours.

Truncating yourself avoids the error, but whatever comes after the cut is lost. So long documents are split into **chunks**, and each chunk is embedded as its own record.

Things to know before using any embedding model

Same model	articles and questions use the same model; changing it means re-embedding everything stored
Matching dimensions	the vector's dimension must match the index's (ours: 1536)
Company jargon	a general model may not know an internal term — add definitions to the text
Languages	cross-language quality varies by model — test Telugu, English and Hinglish
Cost	usage-priced per token — check current pricing

Size	1536 numbers × 4 bytes ≈ 6 KB per vector — about 6 GB per million

Next question: there are many embedding models. **Which one should we use — and how do we decide?**

9 Choosing an embedding model ★

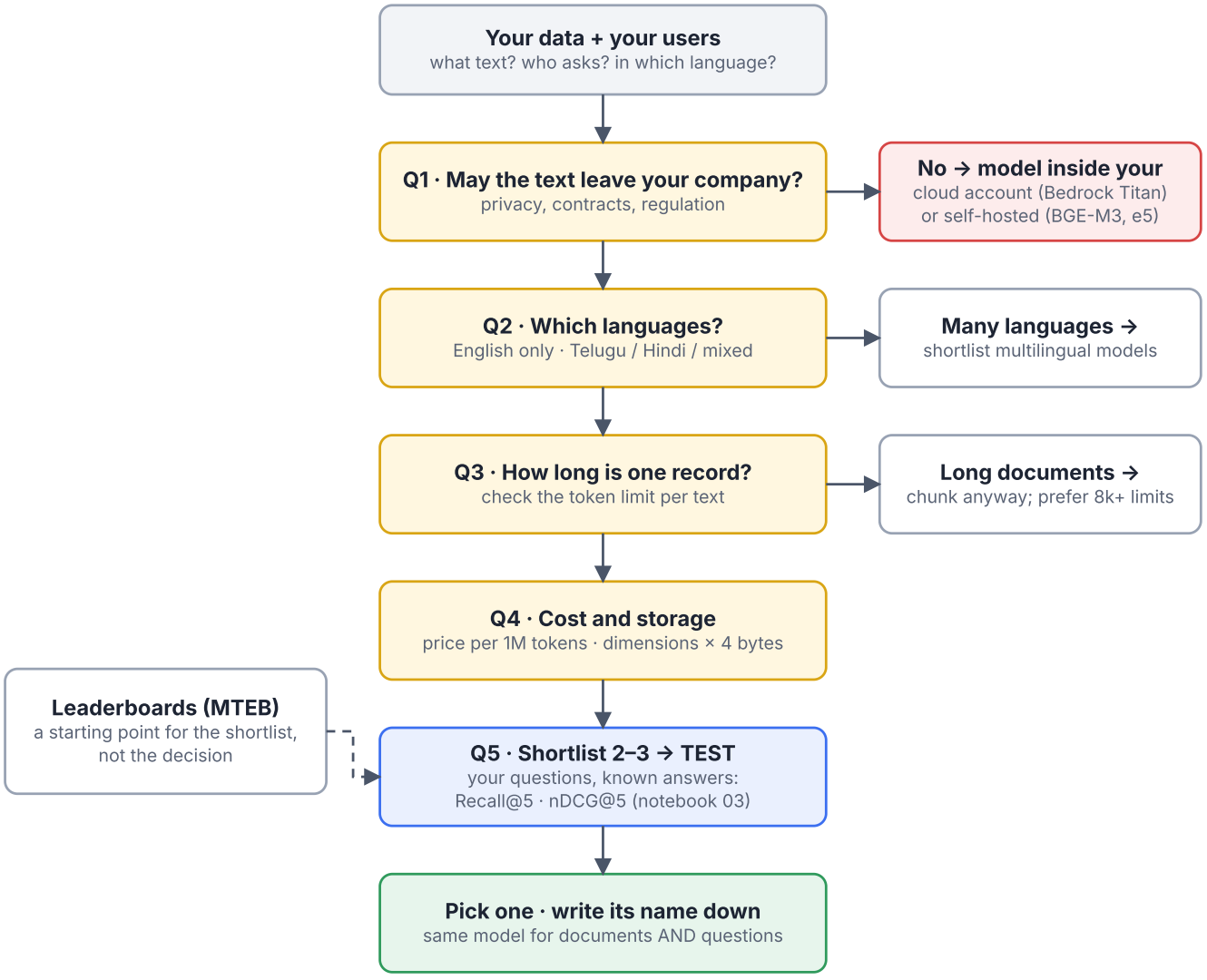


Figure 10. Choosing an embedding model: requirements narrow the list; a test on YOUR questions decides.

Every later step depends on this choice: the index's dimension, what the search can find, the cost, and where your data goes. And it is expensive to change — a new model means **re-embedding everything** and often a **new index**. So choose deliberately, in this order.

Step 1 — Answer five questions about your situation

#	Question	Why it matters	Our course
Q1	**May the text leave the company?**	API models receive your text. If that isn't allowed, use a model inside your own cloud account or run an open model on your own servers	yes — sample data

#	Question	Why it matters	Our course
Q2	**Which languages do users write in?**	English-only models can miss Telugu or Hindi questions; multilingual models are built for them	English workbook; Telugu/Hinglish questions to test
Q3	**How long is one record?**	a text over the limit fails or is cut (Figure 9); long documents need chunking either way	short — a few hundred tokens at most
Q4	**What does it cost to embed and store?**	price per 1M tokens to embed; dimensions × 4 bytes per vector to store — and a model change pays it all again	337 records: a fraction of a cent
Q5	**Which one finds the right records for MY questions?**	the only question that really decides — answered by a test, not by a leaderboard	notebook 03 measures it

Step 2 — Build a shortlist from the candidates

Model	Runs where	Dimensions	Max input	Languages	Price / 1M tokens
OpenAI `text-embedding-3-small`	OpenAI API	1536 (can shorten)	8,191	test yours	$0.02
OpenAI `text-embedding-3-large`	OpenAI API	3072 (can shorten)	8,191	test yours	$0.13
Amazon Titan Text Embeddings V2	AWS Bedrock, in your account	1024 · 512 · 256	8,192	English; 100+ in preview	$0.02
Cohere Embed v4	Cohere API, Bedrock, Azure	256 – 1536	128,000	multilingual	$0.12 (Bedrock)
Google `gemini-embedding-001`	Gemini API, Vertex AI	3072 (768, 1536)	2,048	100+	$0.15
BAAI `bge-m3`	your own servers (MIT)	1024	8,192	100+	your hardware
`multilingual-e5-large`	your own servers (MIT)	1024	512	~100	your hardware
`all-MiniLM-L6-v2`	your own servers	384	256	English	your hardware

Figures as published by each provider in November 2026 — **prices and limits change; check before you budget.** Things the table can't show:

- **Amazon Titan V2** lets you pick a smaller vector: AWS reports 512 dimensions keep about 99% of the accuracy of 1024, and 256 about 97% — a quarter of the storage. AWS also notes cross-language queries return less good results.
- **`multilingual-e5-large`** must be given `"query: "` in front of questions and `"passage: "` in front of documents — forget it and quality drops, with no error.
- **Gemini on Vertex AI** cuts text past 2,048 tokens silently unless you switch that off.

- **Self-hosted** models cost no per-token fee and your text never leaves — but you run and pay for the servers.
- **MTEB** — the Massive Text Embedding Benchmark — ranks models across many public tasks and languages. Use it to build the shortlist; its authors found no single model is best at every task.

Step 3 — Test the shortlist on your own questions

```
model A:  embed the records ──► ask ~900 questions with known answers ──► Recall@5 · nDCG@5
model B:  embed the records ──► ask the same questions                ──► Recall@5 · nDCG@5
then:     compare quality, cost and size ──► pick one
```

Notebook 03 does exactly this: change `EMBED_MODEL`, run again, compare. A leaderboard tells you how a model does on other people's text; this tells you how it does on **yours**.

Step 4 — Our choice for this course, and why

Requirement	`text-embedding-3-small`
Text may leave the company?	yes — training data, not customer data
Language of the knowledge	English — Telugu and Hinglish questions are tested in chapter 9
Record length	far below 8,191 tokens
Cost	$0.02 per 1M tokens — the whole course costs well under a cent
Simplicity	one API key, 1536 dimensions, works with Pinecone's free tier

When we would choose differently: a company whose data must stay in AWS → Titan Text Embeddings V2 in Bedrock (1024 dimensions — a new index and a full re-embed). Customers mainly writing in Telugu or Hindi → shortlist multilingual models and let notebook 03 decide. Whatever you pick: **write the model name down, use it for documents and questions, and store it with each record.**

10 Chapter 7 — Cosine similarity

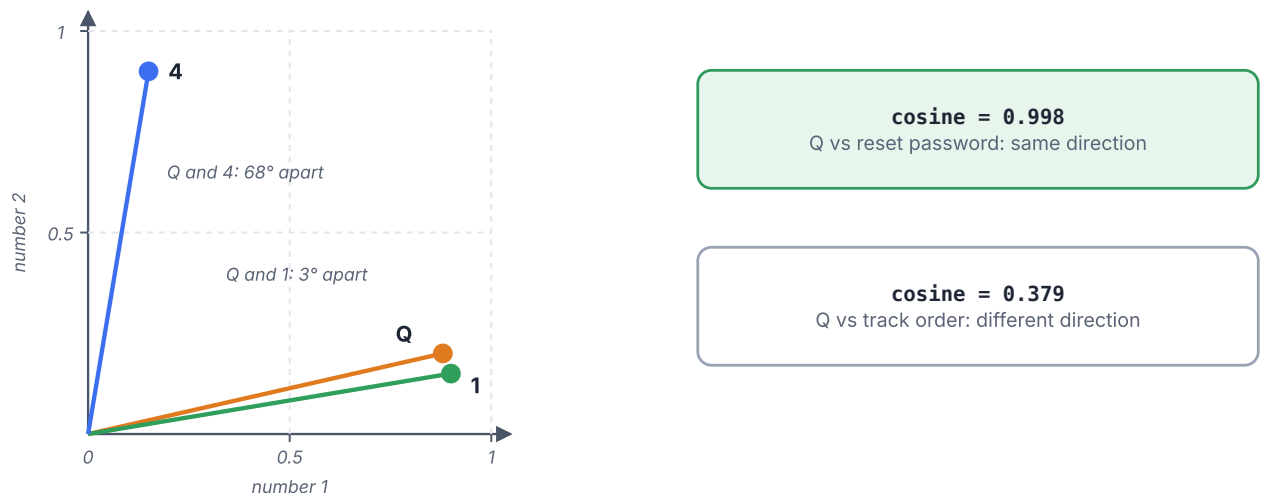


Figure 11. Each embedding as an arrow. A small angle = similar meaning = a high score.

Imagine each vector as an arrow starting at zero. **Cosine similarity compares directions**:

Cosine score	Geometric meaning
`1`	same direction
`0`	perpendicular
`-1`	opposite directions

```
cosine similarity = (A · B) / (length of A × length of B)
```

Students don't calculate this by hand — the libraries and Pinecone do. For text embeddings, a higher score often means a closer match **within that model and search setup**; it is not a universal measure of meaning.

> Real scores bunch together — a good match may be 0.6, not 0.99. A score is **not a probability**: 0.80 doesn't mean "80% sure". The top result is the closest found, not a guarantee it is correct.

> **Next question:** how do we store many embeddings and search them efficiently?

11 Chapter 8 — The vector database

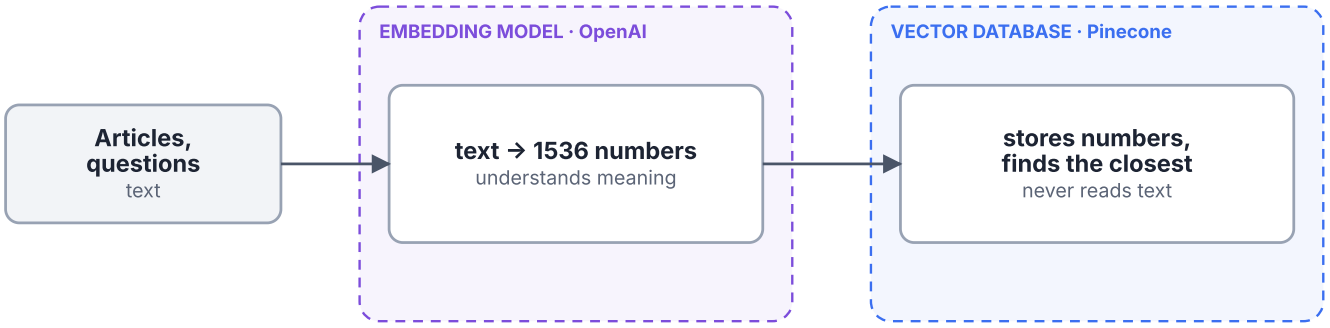


Figure 12. Two different jobs. The model makes the numbers; the vector database stores and compares them.

Why not store embeddings in the relational table from chapter 1? A table can hold a list of numbers, but its normal indexes are built for exact lookups such as `WHERE id = 1`. Finding the **closest** of millions of vectors needs an index built for that job — in a vector database, or an extension such as `pgvector`.

Embedding model	Vector database
converts text into vectors	stores vectors and their records
generates the numbers	searches for the nearest vectors
OpenAI `text-embedding-3-small`	Pinecone

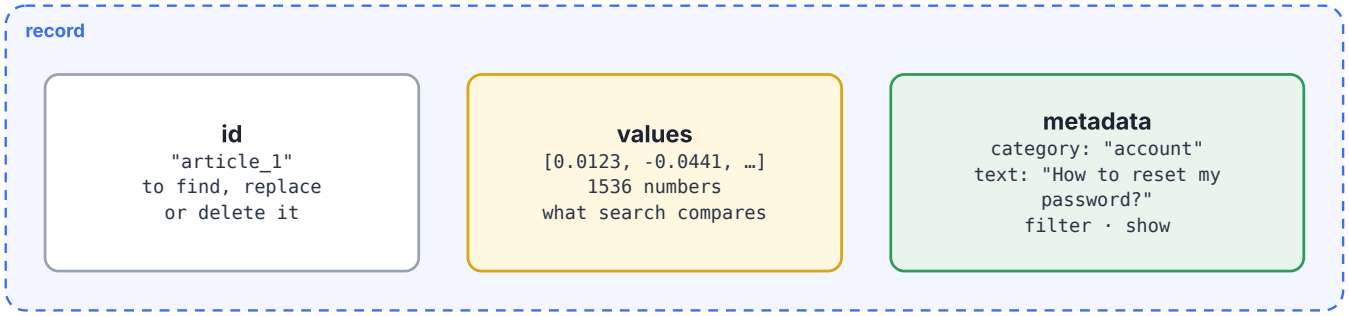


Figure 13. One Pinecone record: id, values (the embedding), metadata.

Field / term	Meaning
`id`	unique identifier for the record
`values`	the embedding used for similarity search
`metadata`	extra information — category, source, the original text
Index	the vector collection; dimension and metric are fixed when it is created
Namespace	a logical section of an index — e.g. `help-articles`, `hr-documents`
Upsert	insert, or replace if the same id already exists
Metadata filter	search only among records that meet a condition, e.g. `category = "account"`

In this notebook the original text goes into metadata so results can show it. Other systems keep the text elsewhere and store a reference.

12 Chapter 9 — Back to the customer

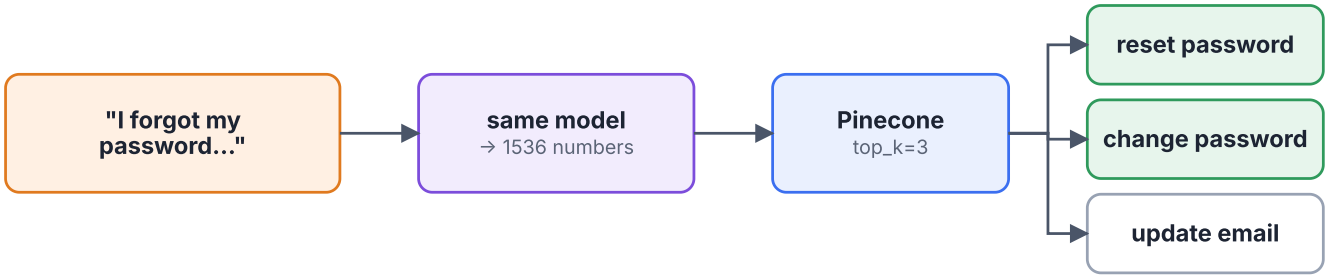


Figure 14. Back to the customer: the same question, now answered.

The same question the keyword search couldn't answer, embedded with the **same model**, sent to Pinecone:

Question	Keyword search (chapter 2)	Vector search (chapter 9)
I forgot my password...	0 rows	password articles should lead
I can't log in anymore	may miss	password articles should lead
I'm locked out of my account	may find the wrong article	password articles should lead

Then `top_k=1` (only the best) and a **billing** filter — which returns the best billing article even though it is a poor match. **A vector database returns the closest records under the search settings; it doesn't know whether they answer the question.** Real systems add score thresholds, reranking and other checks.

Students finish by predicting, then running: "I want to stop paying every month", "Where is my parcel?", and the password question in **Telugu** and **Hinglish** — a live test of Q2 from the model choice.

13 The code, cell by cell

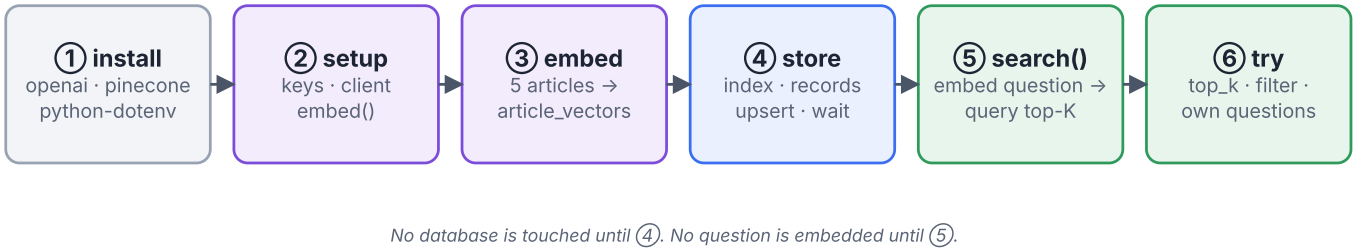


Figure 15. The notebook's six code cells and what each one passes to the next.

Cell	What it does	Passes on	Does NOT
① install	installs the three libraries	—	touch any API
② setup	loads `.env`, checks `OPENAI_API_KEY`, creates the client, defines `embed(texts)`	`client`, `EMBED_MODEL`, `embed`	connect to Pinecone (that waits for ④)
③ embed	one call turns 5 titles into 5 vectors; prints dimension + first 5 numbers	`article_vectors`	store anything
④ store	checks the Pinecone key, creates the index once (1536 · cosine), builds 5 records, upserts, waits	`index`, `NAMESPACE`	call OpenAI again
⑤ search	`search(question, top_k, category)`: embed → query → print score · id · text	the `search` helper	look outside `help-articles`
⑥ try	`top_k=1`, a billing filter, students' own questions	—	change stored records

```
def embed(texts):
    response = client.embeddings.create(model=EMBED_MODEL, input=texts)   # one call, many texts
    return [item.embedding for item in response.data]                     # same order as the input
```

`EMBED_MODEL` is defined **once** and used for articles and questions — the code-level form of "same model everywhere".

14 Key takeaways and knowledge check

The story in one breath: a keyword search can miss a relevant article when the wording differs → we want **similarity search** → which compares numbers → a **vector** is a list of numbers → an **embedding model** turns text into a vector → **cosine similarity** compares vectors → a **vector database** stores vectors with metadata and retrieves the closest → we inspect results, because closest isn't always correct. **The model is chosen by requirements, then by a test on our own questions.**

#	Question	Answer
1	What is a scalar?	one number, such as `30`
2	Dimensions of `[10, 20, 30]`?	3
3	Who generates an embedding?	the embedding model
4	Does one dimension have a readable meaning?	no — meaning is spread across the vector
5	What does `top_k=3` return?	up to the three closest records
6	Embedding model vs vector database?	the model makes vectors; the database stores and searches them
7	The three parts of a Pinecone record?	`id`, `values`, `metadata`
8	Is the top result always correct?	no — it is the closest found
9	Why split long documents?	token limits (an error or a silent cut) and one topic per chunk
10	Why the same model for documents and questions?	each model makes its own numbers; mixing them is meaningless
11	Name the five questions for choosing a model.	data may leave? · languages · record length · cost and size · which finds MY records (tested)
12	Why not just pick the top of a leaderboard?	it measures other people's text; only a test on your questions shows what works for you

Next — 02: the same index and model, now with a real knowledge base for an AI SQL assistant.